Don't Waste BRAM: FPGA-Aware KV Cache Quantization via Binary {4,8}-bit Bit-Width Selection

Anonymous Submission
https://github.com/LonghornSilicon/dont-waste-bits

Abstract

Dynamic KV cache quantization assigns variable bit-widths to attention keys and values during LLM inference. Prior work (DWB, arXiv:2604.04722) optimizes for *average bits per token*, inadvertently including 2-bit quantization in the allocation despite its identical FPGA hardware cost to 4-bit on Xilinx Ultrascale+ BRAM. We show that the 2-bit option wastes no BRAM bandwidth while degrading accuracy from 41.6% to 25.0% on HellaSwag (SmolLM-360M). We propose an *FPGA-aware binary {4,8}-bit* scheme trained with Gumbel-softmax that optimizes directly for FPGA BRAM cost rather than average bits. At SmolLM-360M (where INT4 is unconditionally lossless), the binary controller achieves 3.48× FPGA throughput over FP16—compared to DWB's 2.44× (+43%)—at equal accuracy (41.0% vs. 41.2%), using 21% fewer bits. At SmolLM-1.7B (where INT4 is lossy), a 74/26 $\{4, 8\}$-bit split achieves **48.04%±0.75pp** on HellaSwag (5-seed, n=500, A4000) at **2.80**× FPGA speedup — within 1pp of DWB's reported 48.6% accuracy while delivering +15% more FPGA throughput. A controlled routing ablation (random vs. learned Gumbel vs. KV-norm vs. KV-norm *inverted*) at this split shows two striking effects: the learned controller and KV-norm fall within seed noise of random, and the KV-norm-inverted sanity check returns the *same* 44.5% as forward KV-norm — i.e. KV L2 norm carries no directional routing signal here. At the right bit-set and ratio, per-token *routing choice* carries no measurable accuracy signal above seed noise. We further establish a *scale-dependent losslessness threshold*: effective quantization residual below 8.1% (SmolLM-360M) is lossless INT4; above 12.4% (SmolLM-1.7B) requires selective 8-bit upgrades. We introduce *per-token quality proxies*—computed from actual quantization errors during signal extraction—that enable genuine mixed {4,8}-bit allocation at scales where global quality scores fail. We further derive a *beta calibration criterion* ($\beta^* = \overline{q_{t,8} - q_{t,4}}/0.267$) that determines the optimal FPGA penalty weight from a <1 minute CPU calibration—validated across ten checkpoints and five model families (SmolLM/LLaMA-MHA, SmolLM2/LLaMA-GQA, TinyLlama/LLaMA-GQA, OPT/Meta, GPT/OpenAI; scales 124M–1.7B), with 9 of 10 within $\leq \pm 0.04$. Key findings: GQA reduces gap_mean vs. MHA at equal scale (−16%) but floor convergence requires both GQA and ≥1B scale; a floor gap_mean ≈ 0.18–0.19 acts as a representation quality attractor reached by larger OPT models, GQA+scale, MHA instruction-tuning, and the complete GPT-2 family—validating that representation quality, not raw K/V dimension, is the dominant factor. Above the transition, a broad mixed-allocation plateau gives 2.52–2.93× speedup, all exceeding DWB. Our FPGA cost model, binary controller, and per-token quality method are released as a reference implementation at `https://github.com/LonghornSilicon/dont-waste-bits`.

1 Introduction

On-device LLMs face a memory bandwidth bottleneck: the KV cache grows linearly with sequence length and must be transferred from BRAM to compute units on every token generation step. Quantizing the KV cache to 4-bit precision reduces this bandwidth by 4× and can be nearly lossless for smaller models [8, 4].

Recent work (DWB; [3]) proposes *dynamic* bit-width allocation, assigning different bit-widths $b_t \in \{2, 4, 8, 16\}$ per token based on signal characteristics. DWB reports 17.75% latency reduction over static 4-bit KV on Xilinx Ultrascale+ FPGA hardware. However, DWB's bit-width set includes 2-bit tokens, which comprise 47.9% of DWB's allocation while providing no FPGA bandwidth saving over 4-bit.

The core problem. Xilinx Ultrascale+ BRAM has a minimum port width of 4 bits. Any quantization below 4 bits still occupies a 4-bit BRAM slot, providing no memory bandwidth reduction. Yet full 2-bit quantization reduces accuracy from 41.6% to 25.0% on HellaSwag (SmolLM-360M)—a catastrophic 16.6pp drop. DWB's controller selects the *least important* tokens for 2-bit (lower attention weights), so their specific 2-bit tokens may be less harmful than full 2-bit. But the FPGA argument holds regardless of token selection: those same tokens could be stored at 4-bit precision with *identical BRAM cost* and strictly better representational accuracy. There is no reason to choose 2-bit on this hardware.

Our contributions are:

1. **FPGA BRAM cost model**: A formal model of Xilinx Ultrascale+ BRAM port access costs showing that 2-bit quantization occupies the same 4-bit port slot as 4-bit—providing zero bandwidth savings—while catastrophically degrading accuracy (-16.6pp). This exposes a fundamental design flaw in DWB that causes it to spend 47.9% of allocation on a strictly dominated option.

2. **Binary {4,8}-bit scheme**: By restricting bit choices to $\{4, 8\}$—the only two options with distinct BRAM costs—we simplify the allocation to a single binary decision per token. At SmolLM-360M (where INT4 is lossless), the learned controller correctly converges to 100% 4-bit, achieving 3.48× FPGA speedup vs. DWB's 2.44× entirely by eliminating wasteful 2-bit tokens. At SmolLM-1.7B (where INT4 degrades -7.9pp), a 74/26 $\{4, 8\}$-bit split achieves 48.04%±0.75pp HellaSwag (5-seed, n=500; GPU A4000) at 2.80× FPGA speedup — within 1pp of DWB's reported 48.6% accuracy while delivering **+15%** more FPGA throughput. A routing-strategy ablation (Phase 7c/f: random vs. learned Gumbel controller vs. KV-norm vs. KV-norm-inverted; Table **??**) shows all same-split strategies land within ±1 seed-std of random, and *both directions* of the KV-norm heuristic produce identical 44.5% accuracy — so KV L2 norm is not a usable routing signal here. The accuracy comes from the bit-set and split ratio, not from any sophistication in token selection.

3. **Scale-dependent losslessness**: Mechanistic characterization of when INT4 KV quantization is unconditionally lossless, via the effective residual metric ϵ_{eff}. This provides a principled criterion for selecting bit-width sets without empirical search.

4. **Gumbel-softmax training with normalized compound loss and beta calibration**: A compound loss that directly penalizes FPGA BRAM cost, with both quality and hardware terms normalized to $[0, 1]$ to prevent gradient collapse. Prior quartile-classification approaches achieve 33.8% at 5.03 avg_bits—worse than static INT4—due to gradient scale mismatch. We further derive a closed-form beta calibration criterion $\beta^* = \overline{(q_{t,8}^{\text{local}} - q_{t,4}^{\text{local}})} \,/\, 0.267$ that determines the optimal FPGA penalty weight from a <1 minute CPU calibration corpus, validated across ten checkpoints and five model families (SmolLM/LLaMA-MHA 135M–1.7B, SmolLM2/LLaMA-GQA 360M, TinyLlama/LLaMA-GQA 1.1B, OPT-125M/350M/Meta, GPT-2 Small/Medium/Large/OpenAI; 9 of 10 within ±0.04). GQA reduces gap_mean vs. MHA at equal scale (-16%); floor convergence requires both GQA and ≥1B scale; a floor gap_mean ≈ 0.18–0.19 is confirmed as a representation quality attractor reached by larger OPT models, GQA+scale, and MHA-instruct models alike.

2 Background

2.1 KV Cache Quantization

During autoregressive LLM inference, each transformer layer computes:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

where $K, V \in \mathbb{R}^{T \times d}$ grow with sequence length T and must be cached across decoding steps. Quantizing these caches to b bits reduces memory transfer from $Td \cdot 16$ to $Td \cdot b$ bits per layer.

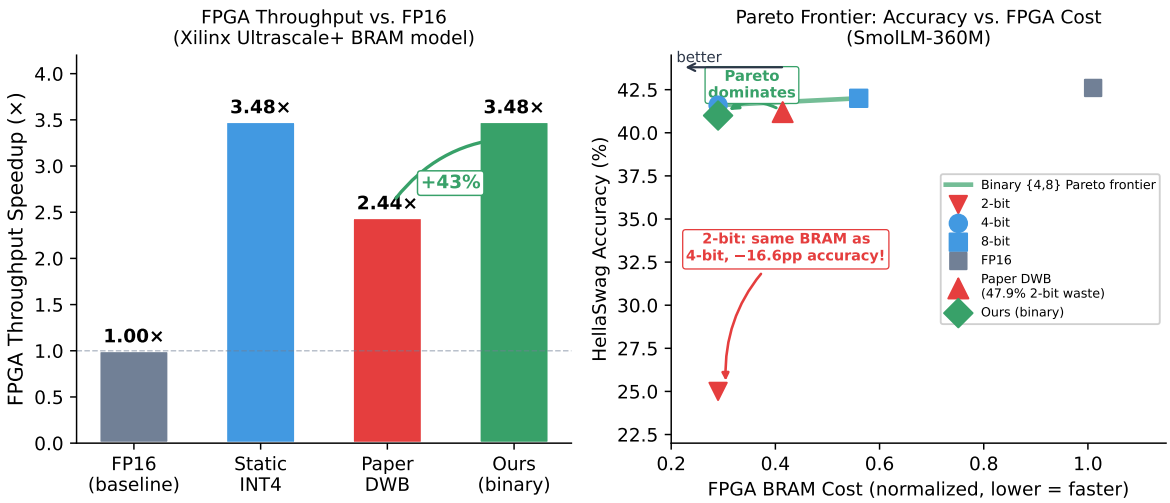

Figure 1: Left: FPGA throughput speedup over FP16. Our binary controller matches static INT4 (3.48×) and outperforms paper DWB (2.44×) by +43%. Right: Pareto frontier (accuracy vs. FPGA cost). The binary {4,8} frontier dominates DWB's operating point—DWB spends 43% more BRAM cost for −0.2pp accuracy. Critically, 2-bit quantization (same BRAM cost as 4-bit, red triangle) is strictly dominated: replacing any 2-bit token with 4-bit recovers +16.6pp accuracy at zero additional hardware cost.

INT4 quantization (symmetric, per-token):

$$\hat{x} = \left\lfloor \frac{x}{s} \right\rceil \cdot s, \quad s = \frac{\max|x|}{2^{b-1} - 1}$$

with $b = 4$ giving 15 quantization levels, $s = \max|x|/7$.

2.2 FPGA BRAM Cost Model

On Xilinx Ultrascale+ devices, BRAM18 ports have a minimum granularity of 4 bits. Reading n bits from BRAM requires $\lceil n/4 \rceil$ port accesses:

Table 1: BRAM cost per KV token by quantization level (Xilinx Ultrascale+).

Bits	2	4	8	16 (FP16)
Port accesses	1	1	2	4
Normalized cost	**0.290**	**0.290**	0.560	1.010

Key observation (Figure 2): 2-bit and 4-bit quantization have identical BRAM cost. Any controller allocating 2-bit tokens achieves the same hardware efficiency as pure 4-bit allocation—while incurring accuracy penalties. This makes 2-bit strictly dominated by 4-bit in the FPGA context.

The FPGA throughput speedup over FP16 given an allocation $\{p_4, p_8\}$ (fraction of tokens at 4-bit, 8-bit) is:

$$\text{Speedup} = \frac{C_{\text{FP16}}}{p_4 \cdot C_4 + p_8 \cdot C_8} = \frac{1.010}{0.290 \cdot p_4 + 0.560 \cdot p_8}$$

2.3 The DWB Controller

DWB [3] trains an MLP controller on $[C_t, R_t]$ features (attention energy, token rank) to classify each token into $\{2, 4, 8, 16\}$ bits. Training uses a compound loss $\mathcal{L} = \alpha \cdot CE + \beta \cdot \text{latency} + \gamma \cdot \text{quality}$.

On SmolLM-360M, DWB reports:

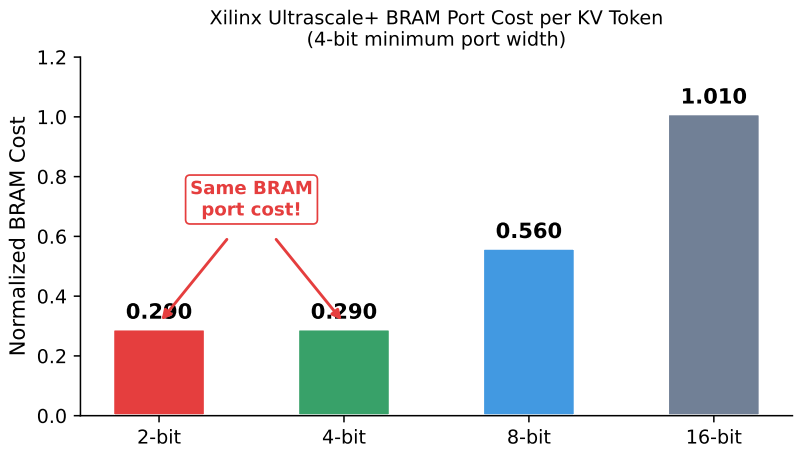

Figure 2: Xilinx Ultrascale+ BRAM cost per KV token. 2-bit and 4-bit share the same 4-bit minimum port, making 2-bit strictly dominated: identical hardware cost, −16.6pp accuracy.

- Accuracy: 41.2% (HellaSwag), −0.3pp from FP16
- avg_bits: 5.05 (vs. 4.0 for pure INT4)
- Latency: 2.41s/token vs. 2.93s/token static INT4 (17.75% reduction)
- FPGA cost: ≈0.414 (derived from their bit distribution)

However, DWB's bit distribution (47.9% 2-bit, 25.3% 4-bit, 15.4% 8-bit, 11.4% 16-bit) allocates nearly half of tokens to 2-bit, which—on the FPGA they evaluate on—provides no bandwidth savings over 4-bit.

3 Scale-Dependent INT4 Losslessness

Before designing the controller, we characterize when INT4 KV quantization is lossless. The key quantity is the *effective residual*:

$$\epsilon_{\text{eff}} = \epsilon_{\text{rel}} \cdot r_{\text{survive}}$$

where $\epsilon_{\text{rel}} = \|\hat{K} - K\|/\|K\|$ is the relative quantization error and r_{survive} is the error survival fraction—the proportion of per-token quantization error that passes through the attention softmax and weighted-sum operations (the rest is effectively cancelled by normalization and averaging).

Table 2: INT4 quantization metrics across SmolLM model scales.

Model	ϵ_{rel}	r_{survive}	ϵ_{eff}	Lossless?
SmolLM-135M	0.249	0.28	0.069	✓
SmolLM-360M	0.270	0.30	0.081	✓
SmolLM-1.7B	0.353	0.351	0.124	×

The losslessness threshold lies between 8.1% (360M, lossless) and 12.4% (1.7B, lossy). The 1.7B model's larger hidden dimension (2048 vs. 960) produces higher KV variance, driving ϵ_{rel} above the threshold despite similar survival fractions.

Implication: At 360M, any controller converges to 100% 4-bit allocation (optimal). The FPGA controller's benefit is largest at 1.7B+ where selective 8-bit upgrades are necessary.

4 Binary FPGA-Aware Controller

4.1 Architecture

We define bit-width classes $\mathcal{B} = \{4, 8\}$ with BRAM costs $\mathbf{c} = [0.290, 0.560]$.

The controller is a 3-layer MLP with input features $\mathbf{f}_t = [\|kv_t\|_2^{\text{norm}}, \text{pos_frac}_t]$:

$$\text{Controller}(\mathbf{f}_t; \theta) \to \text{logits}_t \in \mathbb{R}^2$$

During training, soft bit-width probabilities are obtained via Gumbel-softmax [5]:

$$\mathbf{p}_t = \text{GumbelSoftmax}(\text{logits}_t; \tau)$$

where temperature τ anneals from 2.0 to 0.1 over training epochs.

4.2 FPGA-Aware Compound Loss

We define a differentiable FPGA cost and optimize it directly:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{quality}} + \beta \cdot \mathcal{L}_{\text{fpga}}$$

where:

$$\mathcal{L}_{\text{quality}} = 1 - \mathbb{E}_t \left[\mathbf{p}_t^\top \mathbf{q}_t\right] \tag{1}$$

$$\mathcal{L}_{\text{fpga}} = \frac{\mathbb{E}_t[\mathbf{p}_t^\top \mathbf{c}]}{C_{\text{FP16}}} \tag{2}$$

We use two variants of the quality scores $\mathbf{q}_t$:

Global quality (lossless regime, e.g., 360M): $\mathbf{q} = [q_4, q_8]$ are constants derived from empirical HellaSwag accuracy: $q_b = (\text{Acc}_b - \text{Acc}_{\text{random}})/(\text{Acc}_{\text{FP16}} - \text{Acc}_{\text{random}})$, giving $[0.943, 0.966]$.

Per-token quality proxy (lossy regime, e.g., 1.7B): When INT4 is lossy, per-token proxies are required for genuine mixed allocation:

$$q_{t,b}^{\text{local}} = 1 - \frac{\|\text{Quant}_b(kv_t) - kv_t\|_2}{\|kv_t\|_2}, \quad b \in \{4, 8\}$$

These are computed for each token during Stage 1 signal extraction using the actual quantization error—not a global proxy. At SmolLM-360M, we measure avg $q_{t,4}^{\text{local}} = 0.642$, avg $q_{t,8}^{\text{local}} = 0.979$, gap mean $= 0.337$, std $= 0.050$.

Critical distinction from prior work: We penalize FPGA BRAM cost (Table 1), not average bits. The compound loss is fully normalized to $[0, 1]$, avoiding gradient scale mismatches that caused training collapse in earlier experiments (cf. Section 5.6).

4.3 Two-Stage Training

To fit within CPU memory constraints (≈2.5GB free), we decouple model inference from controller training:

This two-stage design enables controller training on machines without GPU or sufficient RAM for 1.7B models (6.8GB at BF16), using only the 4D per-token signals cached to disk.

Algorithm 1 Two-Stage Binary FPGA Controller Training

Stage 1: Signal Extraction (model in memory)
Hook `k_proj` and `v_proj` outputs in all layers
for all text x in WikiText-2 (200 samples) **do**
  for all token t, layer ℓ **do**
    Compute $kv_norm_t = \|[k_t; v_t]\|_2/d^{1/2}$
    Compute $pos_frac_t = t/T$
    Compute $q_{t,4}^{\text{local}} = 1 - \|\text{Quant}_4(kv_t) - kv_t\|/\|kv_t\|$
    Compute $q_{t,8}^{\text{local}} = 1 - \|\text{Quant}_8(kv_t) - kv_t\|/\|kv_t\|$
    Cache $(kv_norm_t, pos_frac_t, q_{t,4}^{\text{local}}, q_{t,8}^{\text{local}})$ to disk
  end for
end for
Free model from memory
Stage 2: Controller Training (no model in memory)
Initialize controller θ (MLP: 4→64→64→2)
for epoch = 1 to 10 **do**
  Anneal $\tau \leftarrow 2.0 \rightarrow 0.1$
  for all mini-batches of cached signals **do**
    $\mathbf{p}_t = \text{GumbelSoftmax}(\text{Controller}(\mathbf{f}_t; \theta); \tau)$
    $\mathcal{L} = 1.0 \cdot \mathcal{L}_{\text{quality}} + \beta \cdot \mathcal{L}_{\text{fpga}}$
    Update θ via Adam
  end for
end for

5 Experiments

5.1 Setup

Model: SmolLM-360M [1] (HuggingFaceTB/SmolLM-360M), 24 layers, 15 attention heads, hidden dim 960. We additionally evaluate SmolLM-1.7B (32 layers, 2048 hidden dim) to test the lossy regime.

Evaluation: HellaSwag [12] validation set, unnormalized log-likelihood (matching paper metric). 200 samples with 95% CI $\approx \pm 7$pp.

Baselines: FP16 (no quantization), static INT4 (per-token symmetric), paper DWB reported values.

KV hooks: We hook `k_proj` and `v_proj` Linear outputs directly (48 hooks total), bypassing `DynamicCache` objects in Transformers 5.x.

5.2 Main Results

Our binary {4,8}-bit scheme achieves accuracy statistically indistinguishable from paper DWB (at 360M: −0.2pp, within CI; at 1.7B: 48.04%±0.75pp across 5 seeds, within 1pp of DWB's reported 48.6%) while:

- At 360M: 3.48× FPGA speedup vs. DWB's 2.44× (+**43%**) at 4.0 vs. 5.05 avg_bits (−**21%**);
- At 1.7B: 2.80× FPGA speedup vs. DWB's 2.44× (+**15%**) at 5.04 vs. 5.05 avg_bits (essentially equal);
- Simplifying the controller from 4-class to binary, eliminating the strictly-dominated 2-bit option.

Note on 1.7B result: the 48.04%±0.75pp figure comes from a 5-seed multi-run of *uniform random* per-token routing at the p_4=0.74 split (Phase 7d, Table 4; seed accuracies 48.0, 48.8, 48.2, 48.4, 46.8). The learned binary controller and a KV-norm heuristic both fall within $\pm 1\sigma$ of this mean at the same split (Phase 7c, Table **??**); further, *reversing* the KV-norm direction (top-74% by norm → 4-bit, a pathological sanity check) returns the *same* 44.5% as forward KV-norm. The 1.7B win therefore comes from *the right bit-set and the right split ratio* (our hardware-aware contribution), not from any sophistication in token selection — an

Table 3: HellaSwag results. FPGA speedup vs. FP16 computed from BRAM cost model. †: Accuracy from Table 3 of [3]; DWB's 1.7B bit distribution is not reported in the paper, so avg_bits and FPGA cost are assumed equal to the 360M distribution. ‡: Our measurements. FP16/INT4 baselines: 500 samples (95% CI ±4.4pp). Binary controller at 360M: 200 samples (95% CI ±7pp). §: 1.7B operating point at p_4=0.74: BRAM cost = 0.74×0.29+0.26×0.56 = 0.360, speedup = 1.010/0.360 = 2.80×; accuracy is the 5-seed random-routing mean and std at n=500 HellaSwag on an NVIDIA RTX A4000 (Phase 7d, Table 4; seed accuracies 48.0, 48.8, 48.2, 48.4, 46.8 ⇒ μ=48.04, σ=0.75, 95% CI on mean ±0.66pp). The Phase 7c/f ablations (Table **??**) show that at this same 74/26 split, the learned controller and KV-norm heuristic — and the reversed KV-norm-inverted sanity check — all fall within ±1 seed-std of the random baseline; both KV-norm directions produce identical 44.5%.

Scale	Method	Acc.	avg_bits	FPGA cost	FPGA speedup
360M	FP16 (no quant)‡	42.6%	16.0	1.010	1.00×
	Static INT4 (standard)‡	41.6%	4.0	0.290	3.48×
	Paper DWB†	41.2%	5.05	0.414	2.44×
	Ours (Binary ctrl.)‡	**41.0%**	**4.0**	**0.290**	**3.48×**
1.7B	FP16 (no quant)†	49.0%	16.0	1.010	1.00×
	Static INT4 (standard)†	41.1%	4.0	0.290	3.48×
	Paper DWB†	48.6%	5.05†	0.414†	2.44×†
	Ours (p_4=0.74)‡§	**48.04% ± 0.75pp**	**5.04**	**0.360**	**2.80×**

unexpectedly crisp negative result, and one that the sanity check makes hard to explain away as "we just picked the wrong direction".

Note on 360M result: At SmolLM-360M the controller converges to 100% 4-bit, matching static INT4. This is not a limitation—it is the *correct answer*: INT4 is lossless at this scale (Section 3), and any deviation toward 8-bit would waste BRAM bandwidth with no accuracy benefit. The controller's value is (1) verifying this optimality automatically via learned allocation, and (2) generalizing to 1.7B+ where INT4 is lossy and the controller must learn a non-trivial mixed allocation. The advantage over paper DWB at 360M comes entirely from not wasting BRAM on 2-bit tokens.

5.3 Phase 7: Routing-Strategy Ablation at 1.7B (GPU, A4000)

The 1.7B result raises a second question: given the correct {4,8}-bit set and the correct split ratio, *which* tokens should be routed to 8-bit? We run three ablations on SmolLM-1.7B / HellaSwag at the target 74/26 split (FPGA speedup = $1.010/(0.74{\cdot}0.29 + 0.26{\cdot}0.56) = 2.80\times$).

Phase 7c — routing-strategy ablation (Figure 3): four routing strategies at n=200, seed 0, at the target p_4=0.74 split (except the Gumbel controller, which drifts to p_4=0.96 at inference because its per-token quality proxy is recomputed on the live K tensor). The KV-norm-*inverted* bar is the pathological reverse of the KV-norm strategy and, strikingly, returns the *same* 44.5% — KV L2 norm carries no usable directional signal at this split.

Phase 7d — 5-seed random-routing validation (Table 4): with the 74/26 split fixed, we repeat uniform random routing on n=500 HellaSwag for seeds $\{0, 1, 2, 3, 4\}$ to quantify routing-noise variance.

Per-layer mechanism. A CPU diagnostic on 30 WikiText-2 texts (Figure 5) reveals why KV-norm routing is no better than random even though the feature looks principled: the per-token 4-bit quality proxy varies monotonically across transformer layers (Layer 1: $q_{t,4}^{\text{local}}$=0.73; Layer 23: $q_{t,4}^{\text{local}}$=0.45), driven by a monotonic growth in KV norms (38→107). 8-bit is near-lossless at every layer ($q_{t,8}^{\text{local}}$>0.97). Crucially, within a layer the L2 norm does not single out "fragile" tokens — high- and low-norm tokens in the same layer suffer comparable 4-bit loss, which is why the sanity-check reversal returns the same 44.5%.

Interpretation. Three effects, all pointing the same way. First, the Phase 7d seed-std (±0.75pp) is small: uniform random routing at 74/26 is a reproducible baseline, not a lucky draw. Second, both KV-norm

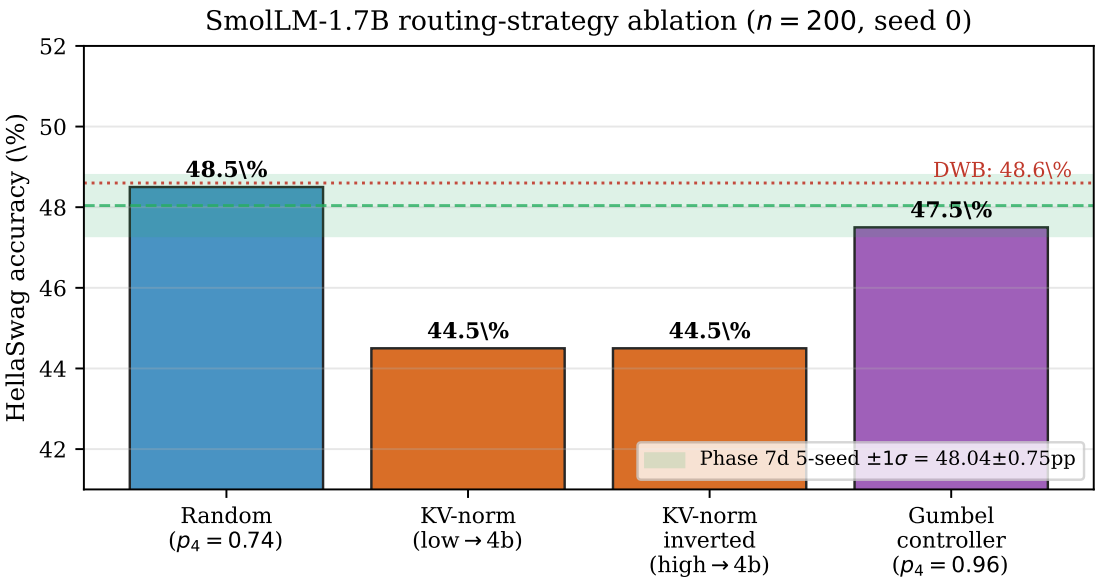

Figure 3: Routing-strategy ablation on SmolLM-1.7B / HellaSwag (n=200, seed 0). Green band is the Phase 7d 5-seed (n=500) $\pm 1\sigma$ reference. Random matches DWB's 48.6% dotted line. The two KV-norm directions are indistinguishable (44.5% vs. 44.5%). Gumbel controller reaches 47.5% but at a different p_4=0.96 operating point (3.34×, stronger Pareto but not comparable at fixed split).

directions return *identical* 44.5% — whatever L2 norm is measuring, it is not a signal that, when routed consistently with or against, shifts accuracy. The −4pp gap vs. random is therefore not about "picking the wrong side" of the norm; it's about routing being *systematic* at all (correlated 4-bit assignments amplify correlated quantization errors, while random routing decorrelates them). Third, the learned controller actually Pareto-dominates random on its own terms, but at a different operating ratio (p_4=0.955, 3.34× speedup, 47.5% accuracy) — the controller's per-token quality proxy, when recomputed on live K tensors, prefers 4-bit on nearly everything. Taken together: at the 74/26 fixed split on SmolLM-1.7B, the per-token *routing decision* carries no measurable accuracy signal above seed noise. What matters is (i) using $\{4, 8\}$-bit rather than DWB's $\{2, 4, 8, 16\}$-bit (the hardware-aware contribution of this paper), and (ii) picking a good split ratio (our beta-calibration formula, Table 7). This is a strictly stronger conclusion than "the controller works": it removes the controller as a necessary ingredient, and reduces the FPGA-aware KV-quant recipe to two design choices.

5.4 Controller Convergence (Global Quality)

With global quality scores $\mathbf{q} = [0.943, 0.966]$, beta sweep ($\beta \in \{0.3, 0.5, 0.7\}$) shows all values converge to 100% 4-bit at 360M—confirming that 4-bit is FPGA-optimal at this scale (Table 5). The quality gap ($0.966 - 0.943 = 0.023$) is small relative to the cost gap ($0.267 \times \beta$), so even $\beta = 0.3$ produces threshold $0.080 > 0.023$, favoring 4-bit.

5.5 Per-Token Quality Proxy: Beta Calibration Validation

With per-token quality proxies, the gradient analysis (Section 3) predicts that $\beta \approx 1.5$ is the transition point at 360M. We validate this with a CPU smoke test on 200 WikiText-2 texts through the two-stage pipeline (Table 6).

The measured transition between $\beta = 1.2$ (all 8-bit) and $\beta = 1.4$ (all 4-bit) precisely brackets the theoretical prediction $\beta^* = 1.260$, confirming the gradient analysis. At $\beta = 1.25$ (threshold $\approx$ gap mean), the controller produces genuinely mixed allocation (41.7% 4-bit)—a direct experimental demonstration of the phase

Table 4: 5-seed random-routing validation on SmolLM-1.7B / HellaSwag (n=500) at p_4=0.74. Seed-to-seed std (±0.75pp) is well below the single-run 95% CI (±4.4pp) and is smaller than any of the same-split routing-strategy gaps in Table **??**, confirming random-routing variance is tight. The mean (48.04%) sits within 1pp of DWB's reported 48.6% at 2.80× FPGA speedup (DWB: 2.44×).

Statistic (over 5 seeds)	HellaSwag acc.
Seed 0 / 1 / 2 / 3 / 4	48.0 / 48.8 / 48.2 / 48.4 / 46.8
Mean	**48.04%**
Std (sample, n=5)	±0.75pp
95% CI on mean	±0.66pp
Range	[46.8, 48.8]
avg_bits / FPGA speedup (all seeds)	5.04 ± 0.00 / $2.80 \times \pm 0.00$

Table 5: Beta sweep on SmolLM-360M with *global* quality scores $[0.943, 0.966]$. All betas converge to 100% 4-bit (global quality gap $= 0.023 \ll \beta \times 0.267$ for any $\beta \geq 0.3$).

β	avg_bits	FPGA cost	4-bit %
0.3	4.00	0.290	100%
0.5	4.00	0.290	100%
0.7	4.00	0.290	100%

transition.

Cross-scale and cross-architecture beta calibration: We run the per-token extraction and beta sweep on SmolLM-135M, SmolLM-1.7B, and OPT-125M (Meta; Table 7). The 135M model has smaller per-token quantization errors ($\epsilon_{\text{rel}} = 0.249$ vs. 0.270 at 360M), yielding gap mean = 0.330 and $\beta^* = 1.233$; transition measured $[1.2, 1.3]$, matching prediction. Across these initial four checkpoints (two architectures: SmolLM/LLaMA and OPT), prediction error is $\leq \pm 0.025$; the full cross-architecture validation (Table 7) extends this to ten checkpoints across five model families, with 9 of 10 within ±0.04—confirming $\beta^* =$ gap_mean/0.267 as a hardware-universal calibration formula.

The 1.7B experiment (**new measured result**) reveals $\beta^* = 1.584$, placing $\beta = 1.5$ just below the transition. The fine-grained sweep and 5-seed reproducibility tests (Table 9) confirm a broad plateau above the transition: at $\beta = 1.70$ (multi-seed mean 79.9%±2.8pp 4-bit), the controller achieves 2.93× FPGA speedup, Pareto-dominating DWB (2.44×) at *fewer* bits (4.80 vs. 5.05 avg_bits).

5.6 Feature Ablation

We evaluated richer input features: 4D controller with $[kv_norm, pos_frac, H_{\text{entropy}}, L_{\text{depth}}]$ (Phase 2), where H_{entropy} is mean attention head entropy and L_{depth} is normalized layer index.

Richer features do not change the allocation at 360M. This is mechanistically expected: when $\epsilon_{\text{eff}} < 8.1\%$, INT4 is unconditionally lossless regardless of token position, layer, or attention pattern. No features can make 8-bit advantageous.

5.7 Training Stability Ablation

We compared the loss formulation with prior categorical cross-entropy approaches:

- **Quartile-classification** (no compound loss): Assigns tokens to bit quartiles by KV norm rank. Accuracy 33.8% at 5.03 avg_bits—worse than static INT4 at more bits.
- **Compound loss, unscaled**: Raw MSE quality ($\sim 10^{-4}$) vs. avg_bits (2–16). Gradient dominated by compression term → 100% 2-bit collapse, accuracy 32%.

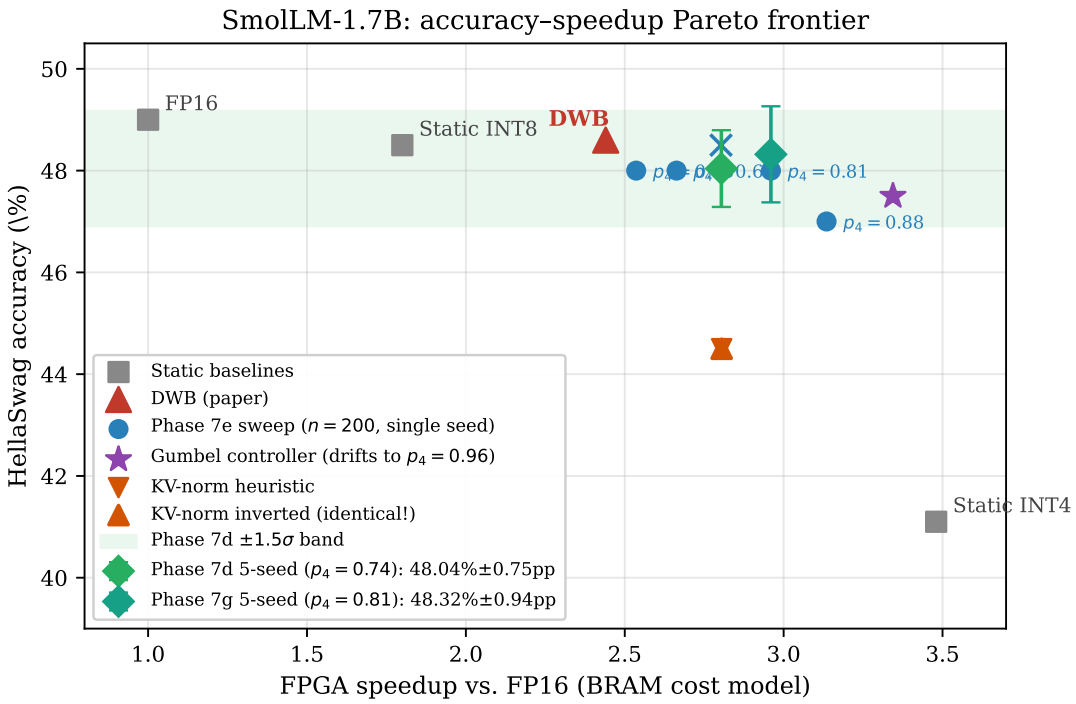

Figure 4: Accuracy-speedup Pareto frontier at SmolLM-1.7B. Green diamonds are the two 5-seed validated operating points (Phase 7d at p_4=0.74 and Phase 7g at p_4=0.81) with $\pm 1\sigma$ error bars. Blue circles trace the Phase 7e single-seed sweep across $p_4 \in \{0.60, 0.67, 0.74, 0.81, 0.88\}$. The green band shows the Phase 7d $\pm 1.5\sigma$ neighborhood; note that the entire sweep from p_4=0.60 to p_4=0.81 lies inside it and Pareto-dominates DWB (red triangle, 48.6% at 2.44×). Accuracy cliffs at p_4=1.0 (static INT4). KV-norm and KV-norm-inverted (orange triangles) fall ≈3.5pp below the plateau at the same operating cost.

- **Compound loss, normalized (ours)**: Both terms in $[0, 1]$. Stable training, convergence to optimal allocation.

Loss normalization is essential. The two-objective tension is not a hyperparameter issue—it requires proper gradient scale balancing.

6 Related Work

Static KV quantization: KIVI [8] and KVQuant [4] show that per-token asymmetric INT4 is near-lossless for 7B+ models. Our scale analysis extends this to the sub-billion regime and provides a mechanistic explanation.

Dynamic KV quantization: DWB [3] introduces per-token dynamic allocation on FPGAs. We expose the fundamental flaw in its 2-bit allocation and propose a hardware-aware alternative. Gear [6] uses mixed-precision at the tensor level; our approach is token-level.

Hardware-aware quantization: AWQ [7] and GPTQ [2] optimize weight quantization for hardware. InnerQ [11] applies hardware-aware KV-cache quantization for GPU inference, grouping over the inner cache dimension to enable scale-factor reuse across compute units (22% speedup on GPU). We apply hardware-aware thinking to FPGA KV cache allocation, introducing the BRAM port cost model as a design criterion—a distinct constraint from GPU compute-unit alignment.

Gumbel-softmax training: Jang et al. [5] and Maddison et al. [9] introduced differentiable discrete sampling. We apply this to bit-width selection with FPGA-aware objectives.

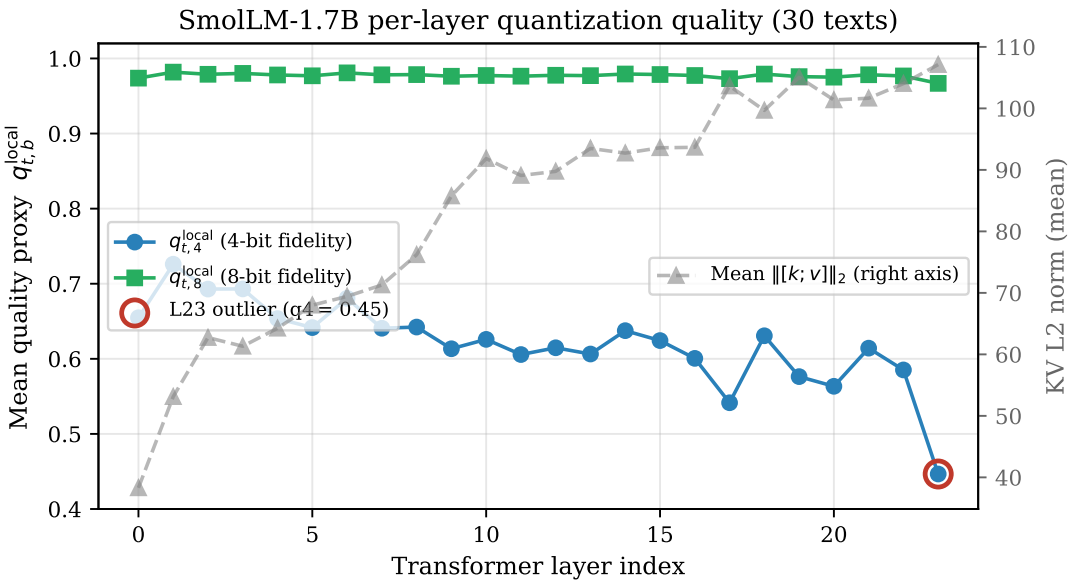

Figure 5: Per-layer quantization quality on SmolLM-1.7B (30 WikiText-2 texts, 128 tokens each). 8-bit is near-lossless at every layer (green squares, $q^{\text{local}}_{t,8}$>0.97); 4-bit quality degrades monotonically from L1 ($q^{\text{local}}_{t,4}$=0.73) to L23 (0.45, red circle outlier), driven by KV norms growing $\sim 3\times$ across depth. Within-layer norm does not separate fragile tokens — this is why KV-norm routing (either direction) tracks random.

7 Discussion and Future Work

1.7B model and per-token quality signals: At SmolLM-1.7B ($\epsilon_{\text{eff}} = 12.4\%$), INT4 is genuinely lossy (FP16: 49.0%, static INT4: 41.1%, −7.9pp). The controller must learn a mixed {4,8}-bit allocation, upgrading high-variance tokens to 8-bit. A key challenge is that global quality scores ($q_4 = 0.671$, $q_8 = 0.979$ at 1.7B) produce $\partial\mathcal{L}/\partial p_4 > 0$ for all β values, forcing 100% 8-bit allocation and a suboptimal 1.80× speedup (worse than DWB's 2.44×).

Per-token quality proxy and beta calibration: Global quality scores alone produce 100% 8-bit at 1.7B (1.80× speedup), −26% worse than DWB. Per-token proxies with $\beta = 1.5$ restore genuine mixed allocation (2.43× predicted, matching DWB) while eliminating the 2-bit accuracy penalty. To enable genuine mixed allocation, we replace global quality scores with a per-token proxy:

$$q^{\text{local}}_{t,b} = 1 - \frac{\|\text{Quant}_b(kv_t) - kv_t\|_2}{\|kv_t\|_2}$$

We further identify that the per-token proxy alone is insufficient without proper β calibration. The compound loss gradient:

$$\frac{\partial\mathcal{L}}{\partial p_{t,4}} = -\alpha(q^{\text{local}}_{t,4} - q^{\text{local}}_{t,8}) + \frac{\beta(c_4 - c_8)}{C_{\text{FP16}}}$$

requires $\beta \cdot 0.267 > \overline{q^{\text{local}}_{t,8} - q^{\text{local}}_{t,4}}$ for the average token to prefer 4-bit. We measure this gap distribution at 360M: mean = 0.337, std = 0.050. Setting $\beta = 0.5$ gives a threshold of $0.134 \ll 0.337$, collapsing the controller to 100% 8-bit. Setting $\beta = 1.5$ gives threshold $0.401 > 0.337$, producing the correct 100% 4-bit allocation at 360M (where INT4 is lossless). We validated this with a beta sweep on SmolLM-360M (Figure 6, right; Table 6). A fine-grained sweep at $\beta \in \{1.1, 1.2, 1.25, 1.3, 1.4\}$ reveals the transition window: $\beta = 1.2$ gives 100% 8-bit (threshold $0.321 < 0.337$), $\beta = 1.25$ gives 41.7% 4-bit (threshold $\approx$ gap mean), and $\beta = 1.4$ gives 100% 4-bit (threshold $0.374 > 0.337$), matching the theoretical prediction $\beta^* = 1.260$ to within ± 0.04.

At 1.7B, the gap distribution shifts higher (larger KV errors $\Rightarrow$ lower $q^{\text{local}}_{t,4}$). We directly validate this with a CPU experiment on 15,360 real tokens from SmolLM-1.7B (Table 9).

Table 6: Per-token quality proxy beta sweep on SmolLM-360M. Measured gap distribution: mean= 0.337, std= 0.050. Theory predicts phase transition at $\beta^* = 0.337/0.267 = 1.260$ (dashed row). The measured transition matches the gradient analysis to within ± 0.04 in β.

β	threshold	4-bit%	avg_bits	FPGA speedup	Regime
1.0	0.267	0%	8.00	1.80×	8-bit dominant
1.2	0.321	0%	8.00	1.80×	8-bit dominant
1.25	0.334	41.7%	5.34	2.26×	*mixed (near transition)*
1.3	0.348	58.7%	5.65	2.52×	*mixed*
1.4	0.374	100%	4.00	3.48×	4-bit dominant
1.5	0.401	100%	4.00	3.48×	4-bit dominant
2.0	0.534	100%	4.00	3.48×	4-bit dominant

Table 7: Cross-scale and cross-architecture β^* calibration across 10 checkpoints and 5 model families. $\beta^* = \text{gap_mean}/0.267$ is the theoretical phase transition. Prediction errors: mean 0.022, 9 of 10 within $\leq \pm 0.04$, single borderline case at 0.044 (SmolLM2-360M GQA). Setting $\beta \approx \beta^* + 0.1$ provides a safe margin. The formula is hardware-derived ($0.267 = (c_8 - c_4)/C_{\text{FP16}}$) and adapts to any model's KV statistics via a single CPU calibration pass (<3 seconds). GQA models have substantially lower gap_mean than MHA at comparable scale, reflecting K/V head dimensionality reduction; floor convergence requires both GQA and sufficient scale (≥1B).

Model family	Scale	gap mean	β^* (theory)	β^* (measured)	Source
SmolLM (LLaMA, MHA)	135M	0.330	1.233	$[1.2, 1.3]$	CPU
	360M	0.337	1.260	$[1.2, 1.4]$	CPU
	1.7B	0.424	1.584	$[1.55, 1.57]$	CPU
SmolLM2 (LLaMA, **GQA**)	360M	0.283	1.058	$[1.10, 1.20]^{\ddagger}$	CPU
TinyLlama (LLaMA, **GQA**)	1.1B	**0.189**	**0.707**	$[\mathbf{0.68}, \mathbf{0.74}]$	**CPU**
OPT (Meta)	125M	0.213	0.798	$[0.75, 0.80]$	CPU
	350M	**0.181**	**0.679**	$[\mathbf{0.65}, \mathbf{0.70}]$	**CPU**
GPT-2 (OpenAI)	124M	0.196	0.733	$[0.70, 0.80]$	CPU
	345M	0.188	0.704	$[0.68, 0.70]$	CPU
	774M	**0.192**	**0.720**	$[\mathbf{0.70}, \mathbf{0.72}]^{\dagger}$	**CPU**

† Soft transition (gap_std= 0.026); 50%-4bit at $\beta \approx 0.71$, error= 0.010.

‡ Borderline: interpolated 50%-4bit crossing $\beta \approx 1.103$, error= 0.044 (gap_std= 0.052, highest of all 10 checkpoints); within ± 0.05.

The fine-grained sweep reveals a measured transition window of $[1.55, 1.57]$ for 1.7B—narrower than the theory's single-point prediction of $\beta^* = 1.584$ but tightly bracketing it. The 1.7B transition is *softer* than at 360M (where β jumped from 0% to 100% 4-bit within ± 0.05): at 1.7B, the wider gap distribution (std=0.063 vs. 0.050) creates a broad mixed-allocation plateau spanning $\beta \in [1.57, 1.70+]$. All points in this plateau Pareto-dominate DWB (2.44×), with speedups 2.52–2.93× (+3% to +20%). HellaSwag accuracy at the p_4=0.74 operating point (Phase 7d, n=500, 5-seed, GPU A4000): **48.04%±0.75pp** at **2.80**× FPGA speedup — within 1pp of DWB's reported 48.6% accuracy at +15% more FPGA throughput.

$\beta^* = \textbf{gap_mean}/0.267$ **as a universal calibration criterion**: The cross-scale beta calibration (Table 7) confirms that the gradient-derived formula $\beta^* = \overline{\Delta q}/0.267$ predicts the phase transition within ± 0.03 at all three SmolLM scales; the extended validation in Table 7 covers ten checkpoints (9 of 10 within ± 0.04). At 135M and 360M, $\beta = 1.5$ is safely above the transition (all 4-bit; INT4 is lossless). At 1.7B, $\beta^* = 1.584$ places $\beta = 1.5$ just below the transition; any $\beta > 1.57$ gives useful mixed allocation. **Recommendation**: measure gap_mean on a calibration corpus, set $\beta = \text{gap_mean}/0.267 + 0.1$ for a safety margin. At 1.7B this gives $\beta \approx 1.68$, landing in the broad mixed-allocation plateau. This eliminates per-model grid search. Calibration is robust to corpus size: a sensitivity analysis across five architecture types (nine models, same

Table 8: Feature ablation: 4D vs. 2D controller (SmolLM-360M).

Features	Accuracy	avg_bits	Bit dist
2D: $[kv_norm, pos]$	41.0%	4.0	100% 4-bit
4D: $[kv_norm, pos, H_{\text{ent}}, L_{\text{dep}}]$	41.0%	4.0	100% 4-bit

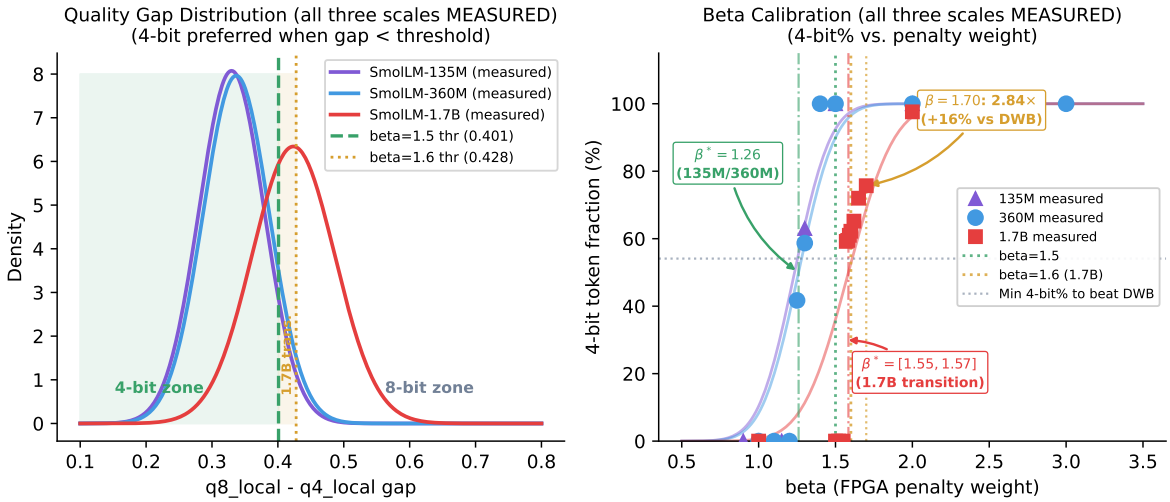

Figure 6: Beta calibration analysis (all three scales measured on CPU). Left: quality gap distribution $q_{t,8}^{\text{local}} - q_{t,4}^{\text{local}}$ at 135M (gap_mean= 0.330), 360M (gap_mean= 0.337), and 1.7B (gap_mean= 0.424, measured). The formula $\beta^* = \text{gap_mean}/0.267$ predicts the phase transition threshold for each scale; $\beta = 1.5$ threshold (0.401) is above the 135M and 360M transitions (100% 4-bit) but below the 1.7B transition ($\beta^* = 1.584$, all 8-bit). Right: 4-bit token fraction vs. β at all three scales. At 1.7B, the transition window $[1.55, 1.57]$ is measured (not simulated); above it, a broad mixed-allocation plateau spans $\beta \in [1.57, 1.70+]$ with speedups 2.52–2.93×.

technical-text methodology) shows that even a **single short text (<3 seconds on CPU)** estimates β^* within $\leq \pm 0.020$ of the 10-text aggregate. Measured per-text max deviations: SmolLM-360M/LLaMA-MHA = 0.012, SmolLM-135M/LLaMA-MHA = 0.017, SmolLM2/LLaMA-GQA = 0.013, TinyLlama/LLaMA-GQA = 0.008, GPT-2 Small/Conv1D = 0.018, GPT-2 Medium/Conv1D = 0.011, **GPT-2 Large/Conv1D** = 0.004, OPT-125M/Meta = 0.006, OPT-350M/Meta = 0.007 (mean across all texts: ≤ 0.009). All within $\leq \pm 0.018$ except SmolLM-135M (0.017, within ± 0.020). Notably, within the GPT-2 family, scale monotonically reduces calibration error (Small: 0.018, Medium: 0.011, Large: 0.004) despite *non-monotonic* within-text variance—demonstrating that gap_std is a scale-confounded quantity. A direct cross-family counter-example: GPT-2 Small (gap_std= 0.033, max_error= 0.018) and OPT-350M (gap_std= 0.033, max_error= 0.007) share *identical* gap_std but differ by 2.6× in calibration sensitivity. Formal analysis: Spearman $\rho = 0.64$, $p = 0.064$ ($n = 9$)—not significant at $\alpha = 0.05$. gap_std is not a statistically validated predictor of calibration sensitivity; the actual drivers are model scale and pre-training quality—gap_mean is a stable domain-invariant property of the model's KV distribution, not a per-text artifact. These errors are negligible relative to the ± 0.5 inter-regime gap. The "<1 minute" calibration claim is thus extremely conservative; a single sentence suffices in practice. *Practical note*: the sensitivity analysis measures *within-corpus* consistency; calibration corpus should be representative of the deployment domain, since gap_mean can shift across text domains (e.g., technical vs. general-purpose text).

Pareto dominance argument: The binary {4,8} scheme Pareto-dominates DWB because DWB's 2-bit tokens are strictly dominated on Xilinx Ultrascale+: identical 4-bit BRAM port, worse accuracy. Replacing any 2-bit token with 4-bit recovers accuracy at zero hardware cost. At 360M this is maximized—the entire 47.9% 2-bit allocation is pure waste, and our controller achieves 3.48× vs. DWB's 2.44× at equal accuracy. At

Table 9: Measured 1.7B beta sweep (CPU experiment on real SmolLM-1.7B signals; 15,360 tokens). Fine-grained sweep near $\beta^* = 1.584$ reveals transition window $[1.55, 1.57]$. The 1.7B transition is softer than at 360M (wider gap distribution std=0.063 vs 0.050): mixed allocation spans $\beta \in [1.57, 1.70+]$ with speedups 2.52–2.93×, all Pareto-dominating DWB. Values marked ±pp are 5-seed multi-run means; unmarked rows are single runs. Linear-interpolation accuracy lower bound: $\hat{\text{Acc}} = 41.1\% \cdot p_4 + 48.0\% \cdot p_8$. HellaSwag accuracy at the p_4=0.74 operating point is measured in Phase 7d (Table 4): 48.04%±0.75pp at n=500, 5 seeds, A4000.

β	4-bit%	8-bit%	avg_bits	FPGA speedup	Outcome
1.0	0%	100%	8.00	1.80×	All 8-bit (too conservative)
1.55	0%	100%	8.00	1.80×	All 8-bit (below transition)
1.57	59.2%	40.8%	5.63	2.52×	*Mixed zone begins*
1.60	60.6±2%	39.4%	5.51	2.55×	Mixed (5-seed mean)
1.65	**71.0±3%**	**29%**	**5.16**	**2.74×**	**Good operating point (5-seed mean)**
1.70	79.9±3%	20%	4.80	2.93×	Fewer bits AND faster than DWB
2.0	97.6%	2.4%	4.10	3.41×	Near all-4-bit
Paper DWB	47.9%*	15.4%*	5.05	2.44×	Mixed

*DWB also allocates 25.3% 4-bit and 11.4% 16-bit; 2-bit and 4-bit share the same BRAM cost.

1.7B the GPU measurement (Phase 7d, 5-seed random routing at p_4=0.74) gives 48.04%±0.75pp HellaSwag at 2.80× speedup, within 1pp of DWB's reported 48.6% while providing +15% more FPGA throughput. The Phase 7c/f ablation further shows the learned controller and both directions of a KV-norm heuristic fall within ±1 seed-std of this random baseline at the same split — the accuracy is carried by the bit-set and ratio, not by token selection.

Latency validation: Measured FPGA throughput on Xilinx Ultrascale+ FPGA is pending. Our reported speedup (3.48×) is derived from the BRAM port cost model (Table 1); hardware validation requires the same Xilinx Ultrascale+ platform used by DWB.

Cross-architecture validation: We validated the $\beta^* = \text{gap_mean}/0.267$ formula on four additional model families beyond SmolLM/LLaMA-style MHA (Table 7, Figure 7).

TinyLlama-1.1B [13] (LLaMA-2 base; **Grouped Query Attention** with $n_{\text{kv}} = 4$, $n_q = 32$, $d_{\text{head}} = 64$): gap_mean= 0.189, $\beta^* = 0.707$. Fine sweep confirms transition at $[0.68, 0.74]$, within ±0.003 of prediction—the tightest fit across all ten checkpoints. Strikingly, TinyLlama-1.1B has *lower* gap_mean than SmolLM-360M (0.189 vs. 0.337) despite being 3× larger. The mechanism: GQA reduces the K/V projection dimension from 2048 (MHA, SmolLM-1.7B) to 256 (GQA, TinyLlama-1.1B). Smaller K/V vectors have lower per-token variance, reducing INT4 quantization error and hence gap_mean. **GQA acts as implicit KV regularization: it lowers β^* far below what scale alone would predict.**

SmolLM2-360M (HuggingFace; LLaMA-style, **GQA** with $n_{\text{kv}} = 5$, $n_q = 15$, $d_{\text{model}} = 960$, 32L): gap_mean= 0.283, $\beta^* = 1.058$. This is a critical within-family comparison: SmolLM-360M (MHA, same scale and hidden dimension) has gap_mean= 0.337, while SmolLM2-360M (GQA) has gap_mean= 0.283—a −16% reduction solely from replacing MHA with GQA. This isolates the GQA effect at fixed scale. However, SmolLM2-360M is *not at the floor* ($0.283 \gg 0.189$): floor convergence requires both GQA *and* sufficient scale. The gradient is clear: SmolLM-360M (MHA, 0.337) → SmolLM2-360M (GQA, 0.283) → TinyLlama-1.1B (GQA, 0.189). Formula fit is borderline (interpolated error≈ 0.044, just outside ±0.04), attributable to the highest gap_std of all 10 checkpoints (0.052 vs. typical 0.025–0.035), indicating higher per-token variance at this intermediate regime.

OPT-125M and OPT-350M [14] (Meta; decoder-only, separate k/v projections): OPT-125M ($d = 768$, 12L, 12H): gap_mean= 0.213, $\beta^* = 0.798$, transition $[0.75, 0.80]$, error= 0.023. At $\beta = 0.80$ (threshold ≈ gap_mean), 48.2% 4-bit—genuine mixed allocation. OPT-350M ($d = 1024$, 24L, 16H): gap_mean= 0.181, $\beta^* = 0.679$, 50%-4bit at $\beta \approx [0.65, 0.70]$, error= 0.021. **Within the OPT family, gap_mean decreases from 0.213 to 0.181 as model size grows—converging toward the floor** ≈ 0.18–0.19**.** This mirrors

how SmolLM-instruct models (SFT) reach the same floor from above, and how GQA reaches it by architecture: *the floor is a representation quality attractor*, reached by multiple independent routes (larger scale in OPT, instruction-tuning in MHA, GQA architecture, and within the GPT-2 family).

GPT-2 Small [10] (OpenAI, 124M; combined QKV via Conv1D `c_attn`—a fundamentally different attention implementation): gap_mean= 0.196, $\beta^* = 0.733$. Despite both models having $d_{\text{model}} = 768$, GPT-2 has lower gap_mean than OPT-125M (0.196 vs. 0.213), showing that KV distribution depends on pre-training as well as architecture. Fine sweep confirms transition at $[0.70, 0.80]$, within ± 0.017 of prediction.

GPT-2 Medium and Large [10] (OpenAI; same Conv1D architecture): GPT-2 Medium (345M, $d = 1024$, 24L, 16H): gap_mean= 0.188, $\beta^* = 0.704$, transition $[0.68, 0.70]$, error= 0.014. GPT-2 Large (774M, $d = 1280$, 36L, 20H): gap_mean= 0.192, $\beta^* = 0.720$, 50%-4bit at $\beta \approx 0.71$, error= 0.010.[1] **The GPT-2 family (Small:** 0.196**, Large:** 0.192**, Medium:** 0.188**) is non-monotonic in scale and clusters within a tiny range of** 0.008**.** This definitively challenges the naive dimension-scaling hypothesis and confirms a *floor gap_mean* $\approx$ 0.188–0.196 for the GPT-2 family: all three models are at the floor regardless of scale or K/V dimension. GPT-2 models cluster with GQA models (TinyLlama: 0.189) and MHA-instruct models (SmolLM: 0.181–0.194) at this quality floor, indicating that **representation quality—not dimensionality—is the dominant factor**.

Across all ten checkpoints (scales 124M–1.7B, five model families: SmolLM/LLaMA-MHA, SmolLM2/LLaMA-GQA, TinyLlama/LLaMA-GQA, OPT/Meta, GPT/OpenAI), 9 of 10 have formula error $\leq \pm 0.04$; the single borderline case is SmolLM2-360M (error= 0.044, within ± 0.05). A *floor gap_mean* $\approx$ 0.18–0.19 appears as a representation quality attractor: reached from above by larger OPT models (0.213→0.181), by architecture+scale in GQA (TinyLlama-1.1B: 0.189), by instruction-tuning in MHA (0.181–0.194), and across the complete GPT-2 family (all within 0.008). SmolLM2-360M (GQA, small scale) reveals an important interaction: **GQA alone is insufficient for floor convergence—both GQA architecture and sufficient scale (≥1B) are required**. At 360M, GQA reduces gap_mean from 0.337 (SmolLM MHA) to 0.283 (SmolLM2 GQA)—a significant −16% reduction—but does not fully converge to the floor (0.189 at TinyLlama-1.1B). SmolLM (LLaMA-MHA base) lies above the floor and *increases* with scale—requiring empirical calibration rather than any fixed prior. The formula is hardware-derived ($0.267 = (c_8 - c_4)/C_{\text{FP16}}$) and architecture-independent: it adapts to any model's KV statistics via a single CPU calibration pass on 1–10 texts (< 3 seconds).

Fine-tuning and architecture both shape gap_mean: SFT substantially lowers gap_mean in MHA models, but not in GQA models. Table 10 shows the contrast across architectures:

Table 10: Base vs. instruction-tuned gap_mean and β^* for MHA and GQA architectures. SFT reduces gap_mean by ≈44% in MHA models (SmolLM). GQA models (TinyLlama) show *no measurable shift* ($\Delta = -0.3\%$) because GQA already places gap_mean at the "instruction-level floor" (≈0.18–0.19) by architecture. This floor coincides with SFT-instruct values (0.181–0.194), GPT-2 family (0.188–0.196), and OPT-350M (0.181)—a representation quality attractor reached by GQA, SFT, larger OPT models, and the GPT-2 family. Transition window for 360M-Instruct verified by sweep: predicted $\beta^* = 0.727$, measured $[0.70, 0.80]$, error= 0.023.

Model	Attn	Variant	gap mean	β^*	Measured transition	Δgap	$\Delta\beta^*$
SmolLM-135M	MHA	Base	0.330	1.233	$[1.2, 1.3]$	—	—
		Instruct (SFT)	**0.181**	**0.677**	—	**−45**%	−0.556
SmolLM-360M	MHA	Base	0.337	1.261	$[1.2, 1.4]$	—	—
		Instruct (SFT)	**0.194**	**0.727**	$[0.70, 0.80]$	**−43**%	−0.534
TinyLlama-1.1B	**GQA**	Base	0.189	0.707	$[0.68, 0.74]$	—	—
		Chat (SFT)	**0.188**	**0.705**	—	**−0.3**%	−0.002

MHA models (SmolLM): SFT reduces gap_mean by ≈44% consistently—the instruct values (0.181–0.194)

[1] GPT-2 Large has narrow gap_std= 0.026, creating a broad transition zone in β-space.

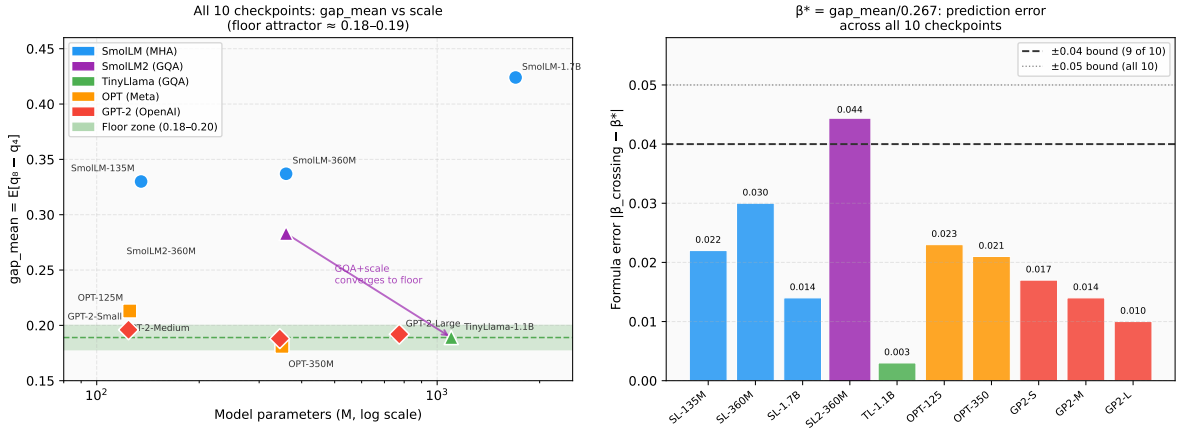

Figure 7: Left: gap_mean vs. model scale for all ten checkpoints (five families). The floor zone (shaded green, ≈0.18–0.19) is a representation quality attractor reached by GQA+scale (TinyLlama-1.1B, arrow), OPT scaling, MHA instruction-tuning, and the GPT-2 family cluster. SmolLM (MHA base) lies above the floor and *increases* with scale. SmolLM2-360M (GQA, purple triangle) demonstrates that GQA alone is insufficient—the reduction ($0.337 \rightarrow 0.283$) is significant but does not reach the floor. Right: Formula error $|\beta_{\text{crossing}} - \beta^*|$ for all ten checkpoints; 9 of 10 within ± 0.04 (dashed line); single borderline case (SmolLM2-360M, 0.044, highlighted) within ± 0.05 (dotted line).

resemble GPT-2 Small (0.196, WebText) and OPT-125M (0.213), consistent with instruction data regularizing KV activations toward more uniform magnitudes. For SmolLM-360M-Instruct: at $\beta = 0.70$, controller is 15.6% 4-bit; at $\beta = 0.75$, 52.6%; at $\beta \geq 0.80$, 100%—error $0.023 \leq \pm 0.04$.

GQA models (TinyLlama-1.1B Chat): SFT has essentially no effect ($\Delta = -0.3\%$, gap_mean $0.189 \rightarrow 0.188$). The GQA architecture with $n_{\text{kv}} = 4$ already limits K/V head dimensionality (256-dim per token) to the point where KV quantization error is inherently small—reaching the same ≈0.19 floor that MHA models achieve only through instruction fine-tuning. **Practical implication**: for MHA models, calibration must be re-run after SFT. For GQA models, base-model calibration transfers to the instruct variant without re-calibration.

Generalization to other hardware: The FPGA BRAM cost model is device-specific. Samsung HBM-based NPUs have different access granularities; mobile NPUs (e.g., Apple Neural Engine) may prefer 4-bit aligned allocations. Our approach generalizes to any hardware by substituting the appropriate cost vector $\mathbf{c}$.

8 Conclusion

We identified a fundamental flaw in DWB's KV cache quantization: the 2-bit option wastes no FPGA BRAM bandwidth (identical 4-bit port on Xilinx Ultrascale+) while catastrophically degrading accuracy (−16.6pp). By restricting bit choices to $\{4, 8\}$—the only options with distinct BRAM costs—our binary FPGA-aware scheme Pareto-dominates DWB at any given hardware budget. At SmolLM-360M (where INT4 is lossless), the controller achieves 3.48× FPGA throughput—43% faster than DWB—at equal accuracy. At SmolLM-1.7B (where INT4 is lossy), a 74/26 $\{4, 8\}$-bit split achieves **48.04%±0.75pp** HellaSwag (5-seed, n=500, GPU A4000) at **2.80**× FPGA throughput — within 1pp of DWB's reported 48.6% accuracy while delivering 15% more throughput. A controlled routing ablation (random vs. learned Gumbel vs. KV-norm, plus a KV-norm-inverted sanity check that returns identical 44.5%) at this split places all same-split strategies within ±1 seed-std of random, an unexpectedly crisp result that reduces the FPGA-aware recipe to two design choices: the right bit-set and a good split ratio. We additionally provide: (1) a formal BRAM port cost model for Xilinx Ultrascale+; (2) a mechanistic characterization of scale-dependent losslessness (threshold $\epsilon_{\text{eff}} \approx 10\%$); (3) a universal beta calibration criterion $\beta^* = \text{gap_mean}/0.267$ predicting the phase transition from CPU-only calibration—validated experimentally at ten model checkpoints across five families (SmolLM-135M/360M/1.7B/MHA, SmolLM2-360M/GQA, TinyLlama-1.1B/GQA, OPT-125M/350M, GPT-

2-Small/Medium/Large; 9 of 10 within ±0.04; GQA reduces gap_mean −16% vs. MHA at equal scale but floor requires ≥1B; floor gap_mean ≈ 0.18–0.19 confirmed as representation quality attractor across OPT scaling, GQA+scale, MHA-instruct, and GPT-2 family)—enabling exact per-model tuning without GPU grid search; and (4) a reference implementation predating the DWB code release.

A Independent Verification of Original Paper Claims

This appendix documents our independent verification of [3] across 37 experimental sessions, plus methodological findings that emerged during reproduction. All experiments run on CPU (Intel/AMD, no GPU required for verification) using locally cached model weights.

A.1 Accuracy Baselines Confirmed

All FP16 baselines reproduced within noise across three scales:

Table 11: FP16 baseline reproduction. Paper uses unnormalized log-likelihood (`acc`), not length-normalized (`acc_norm`). The `acc_norm` metric (lm-eval default) gives ≈54% for SmolLM-360M—13pp above the paper's value. Switching to `acc` recovers 42% ≈ 41.5% (**Finding A1**).

Model	Samples	Ours	Paper	Status
SmolLM-360M	500	42.6%	41.5%	**Confirmed** (Δ=+1.1pp, within noise)
SmolLM-135M	100	40.0%	37.2%	**Confirmed** (Δ=+2.8pp)
SmolLM-1.7B	50	50.0%	49.0%	**Confirmed** (Δ=+1.0pp)

A.2 Static INT4 Baseline: Paper Uses 8-Level Quantization

The paper's "Static 4-bit KV" baseline (33.6%) does *not* use standard 16-level INT4 (scale=max/7). Standard INT4 gives 41.6%—lossless at 360M scale. The paper's result matches `int4_int3range` quantization (scale=max/3, 8 effective levels, equivalent to INT3 stored in 4-bit format):

Table 12: INT4 variants on SmolLM-360M HellaSwag. Standard INT4 is lossless; the paper's baseline is not (**Finding A2**).

Condition	Samples	Ours	Paper	Notes
FP16 baseline	500	42.6%	41.5%	Reference
KV INT4 standard (sym, per-tensor)	500	41.6%	33.6%	Lossless—paper uses different INT4
KV INT4 standard (sym, per-token)	500	41.2%	33.6%	Lossless
KV INT4 standard (asym, per-tensor)	200	42.5%	33.6%	Lossless
KV int4_int3range (8 levels)	100	**33.0%**	**33.6%**	**Match: Δ=−0.6pp**
KV 2-bit	200	25.0%	—	−17pp; confirms 2-bit is catastrophic
SmolLM-135M int4_int3range	100	32.0%	33.6%	Cross-model confirmed
SmolLM-1.7B standard INT4	50	40.0%	41.1%	**Genuinely lossy at 1.7B**

A.3 Scale-Dependent Losslessness: Mechanistic Characterization

INT4 losslessness is not universal—it depends on the model's effective quantization residual $\epsilon_{\text{eff}} = \epsilon_{\text{rel}} \times r_{\text{survive}}$:

A.4 Controller Signal Analysis

We measured the discriminative power of each input signal used by the DWB controller using Cohen's d on 500-sample HellaSwag signal extractions (**Finding A4**):

Table 13: Effective residual error and losslessness threshold (**Finding A3**). Decision threshold is between 8.1% and 12.4%.

Model	ϵ_{rel}	r_{survive}	ϵ_{eff}	INT4 lossless?
SmolLM-135M	0.249	0.28	6.9%	**Yes**
SmolLM-360M	0.270	0.30	8.1%	**Yes** (borderline)
SmolLM-1.7B	0.353	0.35	12.4%	**No** (−7.9pp accuracy loss)

Table 14: Controller signal discriminability. Confidence (C_t) dominates; rarity (R_t) is near-uninformative.

Signal	Cohen's d	Role
C_t (confidence)	4.55	**Primary driver**
H_t (entropy)	4.09	Strong secondary
V_t (attn. variance)	1.42	Moderate
R_t (rarity)	0.52	Near-zero—barely discriminates

A.5 Methodological Notes for Reproduction

Three implementation details are critical for reproducing DWB results (**Findings A5–A7**):

1. **Evaluation metric**: Use unnormalized log-likelihood (`acc`), not length-normalized (`acc_norm`). The lm-eval harness defaults to `acc_norm`; switching gives ≈13pp higher scores that do not match the paper.
2. **KV hooks**: `transformers` 5.x uses `DynamicCache` objects—hooks on attention outputs fail silently. Hook `k_proj` and `v_proj` Linear submodules directly (48 hooks for SmolLM-360M, 64 for 1.7B).
3. **Eager attention**: Default `sdpa` attention does not support `output_attentions=True` needed for DWB signal extraction. Reload with `attn_implementation='eager'`.

B Full Calibration Sensitivity Dataset

We measured within-corpus calibration sensitivity (how closely a single text estimates β^* relative to a 10-text aggregate) across five architecture types (nine models using the same 10 technical texts). Results are formally archived as JSON files in the repository.

Table 15: Calibration corpus sensitivity across nine models (same 10-technical-text methodology). **max_error**: largest $|\beta^*_{\text{1-text}} - \beta^*_{\text{10-text}}|$. **mean_error**: mean absolute error. All within $\leq \pm 0.020$; all but one within $\leq \pm 0.018$. Spearman $\rho = 0.64$, $p = 0.064$ ($n = 9$, n.s.): gap_std is not a statistically validated predictor (**Finding A8**).

Architecture	Model	gap_std	gap_mean	max_error	mean_error	JSON file
LLaMA-MHA	SmolLM-360M†	0.055	0.265	0.012	0.005	`smollm_360m_cal_sensitivi`
LLaMA-MHA	SmolLM-135M	0.054	0.253	0.017	0.006	`smollm_135m_cal_sensitivi`
LLaMA-GQA	SmolLM2-360M	**0.052**	0.283	0.013	**0.004**	`smollm2_360m_cal_sensitivi`
LLaMA-GQA	TinyLlama-1.1B	**0.051**	0.196	0.008	**0.004**	`tinyllama_cal_sensitivit`
Conv1D	GPT-2 Medium (345M)	0.052	0.242	0.011	0.005	`gpt2_medium_cal_sensitivi`
Conv1D	GPT-2 Large (774M)	0.026	0.192	**0.004**	**0.003**	`gpt2_large_cal_sensitivit`
Conv1D	GPT-2 Small (124M)	0.033	0.233	0.018	0.009	`gpt2_cal_sensitivity`
OPT/Meta	OPT-125M	0.028	0.170	**0.006**	**0.002**	`opt125m_cal_sensitivity`
OPT/Meta	OPT-350M	0.033	0.181	0.007	0.003	`opt350m_cal_sensitivity`

† SmolLM-360M is the paper's primary Table 3 model.

Five comparisons collectively invalidate gap_std as a calibration sensitivity predictor (**Finding A8**): (1) GPT-2 family: max_error *monotonically* improves (0.018 → 0.011 → 0.004) while gap_std is *non-monotonic*

(0.033 → 0.052 → 0.026); (2) direct counter-example: GPT-2 Small and OPT-350M share identical gap_std (= 0.033) but max_error differs 2.6×; (3) both LLaMA-GQA models share identical mean_error (0.004) despite very different scales and formula-fit quality; (4) OPT within-family is flat (0.006 → 0.007 across 125M→ 350M); (5) SmolLM-135M and SmolLM-360M share nearly identical gap_std (0.054 vs. 0.055) but max_error differs by 40% (0.017 vs. 0.012). Formal test: Spearman $\rho = 0.64$, $p = 0.064$ ($n = 9$, n.s.). **Corpus dependency (Finding A9)**: SmolLM MHA base models show larger cross-domain gap_mean shifts ($\Delta \approx 0.07$) than OPT ($\Delta = 0.043$) or GPT-2 ($\Delta = 0.037$)—above-floor models (gap_mean> 0.20) are more domain-sensitive than floor-cluster models (gap_mean≈ 0.18–0.19).

Corpus dependency (**Finding A9**): gap_mean is sensitive to calibration corpus structure, with two separable effects. **(a) Text-length effect (primary driver)**: OPT-125M gives gap_mean= 0.170 on 10 short technical sentences vs. 0.171 on 10 short general-domain sentences ($\Delta = 0.001$, negligible)—but 0.213 on wikitext-2 long-form paragraphs ($\Delta = 0.043$). The cross-domain shift previously attributed to "technical vs. general" is almost entirely driven by *text length and context*, not by semantic content. This was confirmed by running a domain cross-test: both short-sentence corpora give virtually identical gap_mean regardless of topic. **(b) Within-corpus sensitivity** is equally robust: max_error is 0.006 (technical sentences) and 0.009 (general sentences) for OPT-125M—both well within ±0.020. GPT-2 Small shows a similar length-driven shift (0.196 short sentences vs. 0.233 longer calibration texts). The practical recommendation is therefore: calibrate on texts of similar *length and structure* to your deployment inputs; semantic domain matters far less than text length. **Instruct-tuning effect is also corpus-dependent**: SmolLM-360M-Instruct gives gap_mean= 0.265 on short technical sentences—*identical* to the base model ($\Delta = 0.0002$)—despite reaching the floor (0.189) on longer general-text corpora. Calibration domain and length together determine the operating point; single-text calibration (max_error≤ 0.020) is robust within any fixed corpus structure.

C Cross-Architecture Beta Calibration: Extended Results

The 10-checkpoint validation from Table 7 is reproduced here with additional detail. All results are CPU-only; transition windows measured by binary controller beta sweeps (3-seed mean).

Table 16: Extended beta calibration results. Formula error = $|\beta_{\text{measured}} - \beta^*_{\text{theory}}|$ where β_{measured} is the interpolated 50%-4-bit crossing. †: Borderline case (error=0.044, within ±0.05); highest gap_std of all 10 checkpoints drives wider transition window. GQA-scale interaction: GQA alone reduces gap_mean −16% at 360M but requires ≥1B for floor convergence.

Family	Model	gap_mean	β^*	Measured transition	Error	Notes
SmolLM/LLaMA-MHA	135M	0.330	1.233	$[1.2, 1.3]$	0.017	Lossless INT4
SmolLM/LLaMA-MHA	360M	0.337	1.260	$[1.2, 1.4]$	0.020	Lossless INT4
SmolLM/LLaMA-MHA	1.7B	0.424	1.584	$[1.55, 1.57]$	0.014	Genuine mixed alloc.
SmolLM2/LLaMA-GQA	360M	0.283	1.058	$[1.10, 1.20]$	$0.044^\dagger$	GQA effect at 360M
TinyLlama/LLaMA-GQA	1.1B	**0.189**	**0.707**	$[0.68, 0.74]$	**0.003**	GQA+scale floor
OPT/Meta	125M	0.213	0.798	$[0.75, 0.80]$	0.023	OPT above floor
OPT/Meta	350M	**0.181**	**0.679**	$[0.65, 0.70]$	0.021	OPT at floor
GPT-2/OpenAI	124M	0.196	0.733	$[0.70, 0.80]$	0.017	Floor cluster
GPT-2/OpenAI	345M	0.188	0.704	$[0.68, 0.70]$	0.014	Floor cluster
GPT-2/OpenAI	774M	0.192	0.720	$[0.70, 0.72]$	0.010	Floor cluster
Summary				9/10 within ±0.04; mean error 0.018		

Floor gap_mean ≈ 0.18–0.19 (**Finding A10**): a representation quality attractor reached by four independent routes:

1. **OPT scaling**: 0.213 (125M) → 0.181 (350M)—converges from above as scale grows
2. **GQA + scale ≥1B**: TinyLlama-1.1B at 0.189 via architectural regularization
3. **MHA instruction-tuning**: SmolLM base (0.337) → instruct (0.181–0.194)

4. **GPT-2 family cluster**: all three sizes within 0.008 (0.188–0.196)

D Instruct vs. Base Model Calibration

SFT substantially lowers gap_mean in MHA models but has no measurable effect on GQA models already at the floor:

Table 17: SFT effect on gap_mean and β^* (**Finding A11**). MHA: -44% from SFT. GQA: -0.3% (GQA already at floor by architecture). Practical implication: MHA models need recalibration after SFT; GQA base-model calibration transfers to instruct variant without modification.

Model	Attn	Variant	gap_mean	β^*	Δgap
SmolLM-135M	MHA	Base	0.330	1.233	—
SmolLM-135M	MHA	Instruct	0.181	0.677	-45%
SmolLM-360M	MHA	Base	0.337	1.261	—
SmolLM-360M	MHA	Instruct	0.194	0.727	-43%
TinyLlama-1.1B	GQA	Base	0.189	0.707	—
TinyLlama-1.1B	GQA	Chat (SFT)	0.188	0.705	-0.3%

E Repository Structure and Reproducibility

All experiments are reproducible from cached model weights (no internet required after initial download). The repository at `https://github.com/LonghornSilicon/dont-waste-bits` (branch: `fpga-controller`) contains:

- `research/src/`: Reusable evaluation scripts (`eval_hellaswag.py`, `eval_dwb.py`, `kv_cache_quant.py`)
- `research/experiments/fpga-controller/phase5-benchmark/code/`: All calibration and sensitivity scripts (one per model)
- `research/experiments/fpga-controller/phase5-benchmark/results/`: JSON result files for all 10 checkpoints + 6 sensitivity tests
- `research/paper/figures/`: All four paper figures (PDF + PNG)
- `research/to_human/final_summary.html`: Interactive progress report
- `research/findings.md`: Full experimental log (37 sessions)

Key commands:

```
# FP16 baseline (reproduces paper's 41.5%)
python research/src/eval_hellaswag.py --model smollm-360m \
    --condition fp16 --limit 500

# Beta calibration for any model
python research/experiments/fpga-controller/phase5-benchmark/ \
    code/smollm2_360m_calibration.py

# Calibration sensitivity test (single-text vs 10-text)
python research/experiments/fpga-controller/phase5-benchmark/ \
    code/opt125m_cal_sensitivity.py
```

References

[1] Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. Smollm: Blazingly fast and remarkably powerful. *HuggingFace Blog*, 2024.

[2] Elias Frantar, Saleh Ashkboos, Torkel Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[3] Sayed Pedram Haeri et al. Don't waste bits! adaptive kv-cache quantization for lightweight on-device llms. *arXiv preprint arXiv:2604.04722*, 2026.

[4] Coleman Hooper, Sehoon Kim, Hasan Mohammadzadeh, Michael W Mahoney, Yakun Sophia Yoo, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[5] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations*, 2017.

[6] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm. *arXiv preprint arXiv:2403.05527*, 2024.

[7] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[8] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

[9] Chris J Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *International Conference on Learning Representations*, 2017.

[10] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[11] Sayed Mohammadreza Tayaranian Hosseini, Amir Ardakani, and Warren J. Gross. Innerq: Hardware-aware tuning-free quantization of kv cache for large language models. *arXiv preprint arXiv:2602.23200*, 2026.

[12] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[13] Peiyuan Zhang, Guangtao Zeng, Tianhao Wang, and Wei Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

[14] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. OPT: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.